

Data Analysis with Python 2024–2025

Parte 4: Pandas (DataFrames na Prática) Conteúdo Teórico

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Aprender os principais conceitos para trabalhar com DataFrames da biblioteca Pandas.
- Saber como criar DataFrames, acessar linhas e colunas e obter estatísticas descritivas.
- Aprender a lidar com valores ausentes.
- Entender como converter e manipular colunas de texto.

Conteúdo

1 O que é um DataFrame

Um DataFrame é a estrutura central da biblioteca pandas para representar dados tabulares: uma grade retangular com linhas e colunas, parecida com uma planilha ou uma tabela de banco de dados. Cada coluna pode ter seu próprio tipo de dado (números inteiros, números decimais, texto, datas, etc.), e tanto as linhas quanto as colunas possuem um índice que permite identificá-las.

Antes de mais nada, os módulos costumam ser importados assim:

```
import pandas as pd
import numpy as np
```

Existem três formas comuns de construir um DataFrame do zero: a partir de uma matriz do NumPy, a partir de um conjunto de colunas, ou a partir de um conjunto de linhas.

1.1 Construindo a partir de uma matriz NumPy

Quando já temos os dados organizados em uma matriz bidimensional, basta informar os nomes das colunas e das linhas (opcionalmente):

```
notas = pd.DataFrame(
    np.random.randn(3, 2),
    columns=["Prova1", "Prova2"],
    index=["Ana", "Bruno", "Carla"]
)
notas
```

O resultado é uma tabela de 3 linhas por 2 colunas, em que cada linha corresponde a um estudante e cada coluna a uma prova. Os rótulos das linhas podem ser consultados com `notas.index`, e os das colunas com `notas.columns`. Quando esses parâmetros não são informados, o pandas atribui automaticamente índices inteiros sequenciais (um objeto do tipo `RangeIndex`, semelhante ao `range` nativo do Python).

1.2 Construindo a partir de colunas

Uma coluna pode vir de uma lista comum, de um vetor NumPy ou de uma Series do pandas. Quando usamos uma Series, o atributo `name` dela pode ser aproveitado como nome da coluna:

```
idades = pd.Series([23, 31, 40], name="idade")
pd.DataFrame(idades)
```

Para combinar várias colunas de uma vez, a forma mais natural é usar um dicionário, em que cada chave vira o nome de uma coluna:

```
salarios = pd.Series([3200, 4100, 5300])
cidades = pd.Series(["Recife", "Curitiba", "Belém"])
pd.DataFrame({"salario": salarios, "cidade": cidades})
```

1.3 Construindo a partir de linhas

Também é possível passar uma lista de linhas ao construtor. Cada linha pode ser um dicionário, uma lista, uma Series ou um vetor NumPy. Se as linhas forem dicionários, as chaves definem automaticamente os nomes das colunas:

```
funcionarios = pd.DataFrame([
    {"nome": "Marina", "cargo": "Engenheira", "salario": 7800},
    {"nome": "Diego", "cargo": "Analista", "salario": 5200}
])
```

Alternativamente, quando as linhas são listas simples (sem chaves), os nomes das colunas devem ser informados explicitamente pelo parâmetro `columns`:

```
funcionarios = pd.DataFrame(  
    [[7800, "Marina", "Engenheira"], [5200, "Diego", "Analista"]],  
    columns=["salario", "nome", "cargo"]  
)
```

Um detalhe importante: quando construímos um `DataFrame` a partir de um dicionário de colunas, a ordem das colunas no resultado segue a ordem das chaves do dicionário nas versões atuais do Python e do pandas. Do ponto de vista da informação contida na tabela, a ordem das colunas é irrelevante – mas frequentemente escolhemos uma ordem específica para tornar a leitura mais natural ou para seguir alguma convenção do domínio dos dados.

2 Acessando linhas e colunas

2.1 O comportamento dos colchetes

Diferentemente de uma matriz NumPy, o operador de colchetes `[]` em um `DataFrame` não aceita diretamente um par de índices (linha, coluna). Apenas uma dimensão pode ser especificada de cada vez – mas qual dimensão, exatamente, depende do tipo do valor usado dentro dos colchetes:

- Se for um nome de coluna (ou uma lista de nomes), os colchetes selecionam colunas.
- Se for um inteiro, o comportamento só funciona quando esse inteiro é, de fato, um rótulo explícito de alguma coluna – caso contrário, ocorre um erro.
- Se for uma fatia (slice) ou uma máscara booleana, os colchetes selecionam linhas.

Veja o exemplo com a tabela de funcionários criada acima:

```
funcionarios["nome"]                # seleciona a coluna "nome" ->  
Series  
funcionarios[["nome", "cargo"]]      # seleciona duas colunas ->  
DataFrame  
funcionarios[0:1]                    # seleciona a primeira linha (  
fatia)  
funcionarios[funcionarios.salario > 6000] # linhas com salário alto
```

Se uma dessas operações devolve uma `Series`, ainda é possível encadear um segundo colchete para chegar a um valor individual:

```
funcionarios["salario"][1]          # repare na ordem: coluna, depois linha  
# 5200
```

Essa forma encadeada funciona, mas não é recomendada como prática geral, especialmente quando combinada com atribuição de valores, pois pode gerar avisos ou comportamentos inesperados no pandas. A seção seguinte apresenta uma alternativa mais clara.

2.2 Os atributos `loc` e `iloc`

Para evitar a ambiguidade descrita acima, o pandas oferece dois atributos de indexação com um comportamento previsível e uniforme:

- `loc`: usa os rótulos explícitos (os nomes que aparecem em `index` e `columns`).
- `iloc`: usa posições inteiras implícitas, exatamente como acontece com arrays NumPy.

Em ambos os casos, a primeira posição dentro dos colchetes refere-se a linhas, e a segunda, a colunas – a mesma ordem usada pelo NumPy.

```
funcionarios.loc[1, "salario"]      # valor na linha de rótulo 1, coluna "
                                     salario"
funcionarios.iloc[-1, -1]          # último valor da última linha (canto
                                     inferior direito)
funcionarios.loc[1, ["nome", "salario"]] # múltiplas colunas para uma
                                     linha
```

Com `iloc`, todas as técnicas de indexação do NumPy funcionam normalmente: índices únicos, fatias, listas de posições (*fancy indexing*) e máscaras booleanas. Com `loc`, a lógica é a mesma, mas usando os rótulos em vez de posições.

Uma boa prática é preferir `loc` e `iloc` sempre que o objetivo for selecionar um elemento específico ou um subconjunto bem definido de linhas e colunas simultaneamente, deixando os colchetes simples para os casos mais triviais (uma coluna, ou um filtro por condição).

3 Visão geral e estatísticas descritivas

Uma vez que os dados estão carregados em um `DataFrame`, o próximo passo natural é obter um panorama rápido do seu conteúdo. O método `describe()` calcula, para cada coluna numérica, um conjunto de estatísticas-resumo (contagem, média, desvio padrão, mínimo, quartis e máximo) e devolve tudo isso organizado em um novo `DataFrame`:

```
clima = pd.read_csv("temperaturas_recife_2023.csv")
clima.describe()
```

Esse método é uma das primeiras ferramentas usadas na fase de análise exploratória, pois ajuda a identificar rapidamente problemas como valores fora da escala esperada, colunas quase constantes ou grande variabilidade nos dados.

Também é possível calcular estatísticas isoladas, como a média de cada coluna, usando `mean()`. Por padrão, essas agregações são feitas ao longo das linhas, produzindo um resultado por coluna – exatamente como ocorre com as funções equivalentes do NumPy:

```
clima_numerico = clima.drop(columns=["ano", "mes", "dia"])
clima_numerico.mean()
```

4 Valores ausentes

É comum que conjuntos de dados reais tenham medições faltando – um sensor que falhou em um determinado dia, um campo que não foi preenchido em um formulário, e assim por diante. O pandas representa esses "buracos" de duas formas, dependendo do tipo da coluna:

- Em colunas numéricas (float), o valor especial `nan` (*Not a Number*, disponível também via `numpy.nan`) é usado.
- Em colunas de outros tipos (como texto), o valor `None` do próprio Python costuma ser usado, e o tipo da coluna passa a ser `object`.

Um detalhe interessante é que o tipo `int` não admite valores ausentes: se uma coluna de inteiros ganha um `nan`, o pandas automaticamente promove essa coluna para o tipo `float`, já que apenas números de ponto flutuante conseguem representar o `nan`.

```
pd.Series([10, 20, 30])          # dtype: int64
pd.Series([10, 20, 30, np.nan])  # dtype: float64 (promovido
                                automaticamente)
```

4.1 Localizando valores ausentes

O método `isnull()` devolve uma máscara booleana do mesmo formato do DataFrame original, indicando célula a célula onde há valores ausentes. Sozinho, esse resultado não é muito prático para filtrar dados – mas combinado com `any(axis=1)`, ele permite encontrar todas as linhas que têm ao menos um valor faltando:

```
clima[clima.isnull().any(axis=1)]
```

O método complementar `notnull()` funciona da forma oposta, sinalizando onde os valores estão presentes.

4.2 Removendo ou preenchendo valores ausentes

Duas estratégias comuns para lidar com dados faltantes são remover as linhas/colunas afetadas, ou preencher os buracos com algum valor razoável.

O método `dropna()` remove linhas (por padrão, `axis=0`) ou colunas (`axis=1`) que contenham valores ausentes:

```
clima.dropna().shape      # remove linhas com algum valor faltando
clima.dropna(axis=1).shape # remove colunas com algum valor faltando
```

Os parâmetros `how` e `thresh` permitem ajustar esse comportamento – por exemplo, exigir que todos os valores de uma linha estejam ausentes para que ela seja descartada, ou definir um número mínimo de valores válidos para que a linha seja mantida.

Já o método `fillna()` substitui os valores ausentes por algo definido pelo usuário, podendo ser:

- um valor constante, passado diretamente como argumento;
- `method="ffill"`, que repete o último valor válido anterior (preenchimento "para frente");
- `method="bfill"`, que usa o próximo valor válido (preenchimento "para trás").

```
clima_preenchido = clima.fillna(method="ffill")
```

Para situações mais sofisticadas, em que se deseja estimar o valor ausente a partir da tendência dos vizinhos, existe ainda o método `interpolate()`, que não será detalhado aqui, mas vale a pena conhecer.

5 Convertendo o tipo das colunas

Às vezes uma coluna é carregada com um tipo diferente do desejado – números interpretados como texto, por exemplo. O pandas oferece algumas ferramentas para essa conversão:

- `Series.map(funcao)`: aplica uma função a cada elemento da série, útil para conversões simples, como `map(int)` ou `map(str)`.
- `pd.to_numeric(serie, errors...)`: converte uma série para um tipo numérico, com controle sobre o que fazer quando a conversão falha (por exemplo, `errors="coerce"` transforma valores inválidos em `nan` em vez de lançar uma exceção).
- `DataFrame.astype(tipo)`: converte de uma vez todas as colunas (ou, passando um dicionário, colunas específicas) para os tipos desejados.

```

pd.Series(["10", "20"]).map(int)           # texto ->
    inteiro
pd.to_numeric(pd.Series([1, "abc"]), errors="coerce") # gera NaN no
    lugar do erro

tabela = pd.DataFrame({"a": [1, 2], "b": [3, 4], "c": [5, 6]})
tabela.astype(float)                       # converte todas
    as colunas
tabela.astype({"b": float, "c": str})      # tipos
    diferentes por coluna

```

6 Processamento de texto

Quando uma coluna contém *strings*, o pandas disponibiliza versões vetorizadas dos métodos de texto do Python por meio do acessador `.str`. Isso significa que é possível aplicar operações como capitalização, divisão de *strings*, substituição de trechos, etc., a todos os elementos da coluna de uma só vez, sem precisar escrever um laço explícito:

```

nomes = pd.Series(["joana silva", "pedro alves", "clara souza"])
nomes.str.title() # capitaliza cada palavra

```

Para separar uma coluna de texto em várias colunas – por exemplo, dividir "nome sobrenome" em duas colunas, usa-se `str.split()` com o parâmetro `expand=True`, que transforma o resultado (que por padrão seria uma lista por célula) em colunas reais de um novo DataFrame:

```

nomes.str.split(expand=True)

```

Sem `expand=True`, cada elemento da série resultante seria uma lista de tokens – útil em alguns contextos, mas menos conveniente quando queremos colunas separadas.

7 Resumo

Os principais pontos apresentados neste capítulo foram:

- Um DataFrame pode ser criado a partir de uma matriz NumPy, de colunas ou de linhas, e cada abordagem tem sua sintaxe própria.
- O comportamento dos colchetes simples (`[]`) depende do tipo do índice usado – por isso, `loc` e `iloc` são preferíveis quando se quer previsibilidade, seguindo a mesma ordem de dimensões do NumPy (linhas, depois colunas).
- O método `describe()` oferece um resumo estatístico rápido, muito útil na análise exploratória inicial.
- Valores ausentes são representados por `nan` (colunas numéricas) ou `None` (colunas de objetos), e podem ser localizados, removidos ou preenchidos com `isnull`, `dropna` e `fillna`.
- Colunas podem ser convertidas entre tipos com `map`, `to_numeric` e `astype`.
- Colunas de texto ganham operações vetorizadas por meio do acessador `.str`, incluindo divisão em múltiplas colunas com `split(expand=True)`.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.